

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



Báo cáo kỹ thuật
Thực nghiệm chuỗi chứng chỉ EJBCA

Học phần: **Nhập môn An toàn thông tin – IT4015**

Mã lớp: 167834

Giảng viên:	PGS. TS. Nguyễn Linh Giang	
Nhóm sinh viên:	Đỗ Trường Giang	20235066
	Hoàng Minh Khôi	20230042
	Phạm Gia Hưng	20230036
	Nguyễn Thanh Tú	20230075

Hà Nội, Ngày 23 tháng 7 năm 2026

1 Mục tiêu và phạm vi

Thực nghiệm dựng một PKI thu nhỏ nhằm chứng thực cho máy chủ **server web.local**, sao cho máy chủ **client** có thể tạo liên kết HTTP tin cậy (HTTPS) với máy chủ. Hạ tầng chứng thực bao gồm Root CA được cài trực tiếp vào kho tin cậy (trust store) của máy chủ client và ký chứng chỉ cho Intermediate CA. Máy chủ RA tiếp nhận, kiểm tra và phê duyệt yêu cầu cấp chứng chỉ máy chủ; Intermediate CA sau đó ký chứng chỉ mà Nginx dùng khi thiết lập kết nối TLS với client.

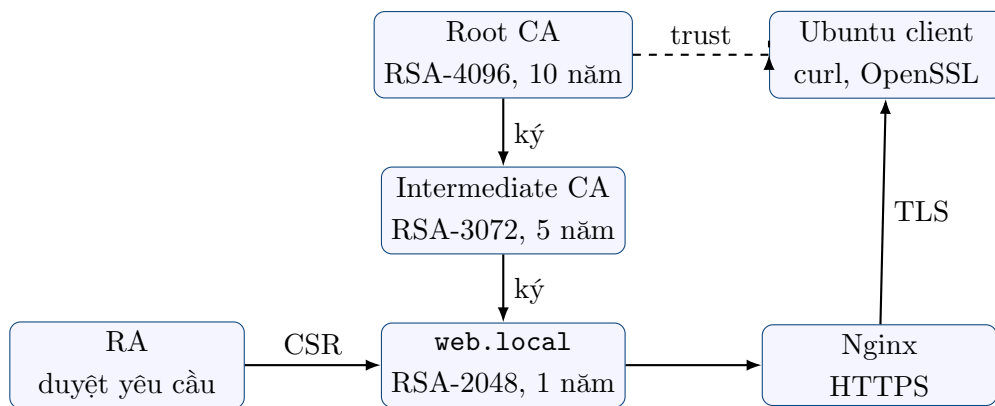
Mô hình tập trung vào hai điều kiện của đường dẫn chứng chỉ. Theo RFC 5280, trường **Issuer** phải nổi đúng với **Subject** của CA phát hành và chữ ký trên mỗi chứng chỉ phải được xác minh khi dựng đường dẫn [1]. Chuỗi hợp lệ được dùng làm đối chứng, sau đó so sánh với hai tình huống lỗi:

1. chuỗi đầy đủ và nguyên vẹn;
2. chuỗi thiếu chứng chỉ Intermediate CA;
3. chứng chỉ **web.local** bị đổi Issuer từ Intermediate CA thành Intermediate CB, nhưng giữ nguyên chữ ký.

Phạm vi báo cáo giới hạn ở quan hệ ký và kiểm tra chuỗi. Xác minh quyền kiểm soát tên miền tự động, VA, CRL, OCSP, thu hồi chứng chỉ và đo hiệu năng không thuộc phạm vi này. Kết quả của thực nghiệm được thể hiện ở bảng đánh giá trong chương 6 và kết luận ở chương 7.

2 Kiến trúc thực nghiệm

Hình ?? mô tả các thành phần và đường đi tin cậy của mô hình.



Hình 1: Các thành phần và quan hệ tin cậy của thực nghiệm

Thực nghiệm sử dụng thư viện mã nguồn mở EJBCA, bản Community. Hạ tầng được đóng gói trong Docker để cung cấp CA UI và RA UI, phù hợp với hướng dẫn triển khai môi trường thử nghiệm của EJBCA [2]. Máy chủ server chạy trên tiến trình Nginx trong một container riêng. Trên máy Ubuntu, tên **web.local** được ánh xạ tới 127.0.0.1 để client truy cập máy chủ bằng đúng tên có trong chứng chỉ.

Root CA là neo tin cậy duy nhất của client. Chứng chỉ Intermediate CA được gán **Basic Constraints: CA=TRUE**, **keyCertSign** và **cRLSign**. Chứng chỉ máy chủ đặt SAN là **DNS:web.local** và cho phép **digitalSignature**, **keyEncipherment** cùng **serverAuth**. Các thuộc tính này tách quyền ký chứng chỉ của CA khỏi mục đích xác thực máy chủ HTTPS.

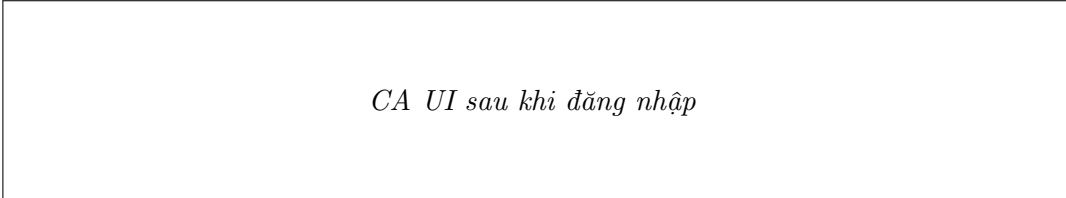
3 Khởi tạo CA và RA

3.1 Mở giao diện EJBCA

EJBCA được khởi chạy trên Ubuntu với các cổng dành cho HTTP và HTTPS. Lệnh dưới đây dùng cho môi trường trình diễn:

```
1 docker run -d --name ejbca-ce \  
2 -p 8080:8080 -p 8443:8443 \  
3 -h ca.local \  
4 -e TLS_SETUP_ENABLED=true \  
5 keyfactor/ejbca-ce
```

Sau khi container khởi động, quản trị viên lấy thông tin SuperAdmin từ log, cài chứng thư quản trị vào trình duyệt rồi truy cập CA UI. Quy trình khởi động, tạo chứng thư quản trị và đăng nhập giao diện được EJBCA mô tả trong hướng dẫn container [2].



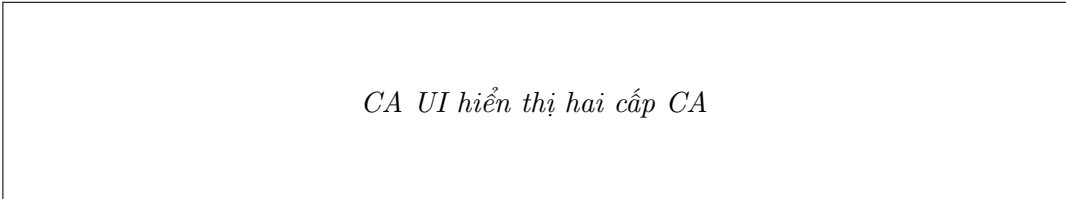
CA UI sau khi đăng nhập

Hình 2: Giao diện quản trị EJBCA của môi trường thực nghiệm

3.2 Tạo Root CA và Intermediate CA

Trong **CA Functions > Certificate Authorities**, nhóm tạo Root CA tự ký bằng RSA-4096, SHA-256 và thời hạn 10 năm. EJBCA yêu cầu chọn **Self Signed** khi Root CA ký chứng chỉ của chính nó [3].

Intermediate CA dùng RSA-3072, SHA-256 và thời hạn 5 năm. Trường **Signed By** trở tới Root CA; profile của CA này cho phép ký chứng chỉ, ký CRL và đặt **pathLenConstraint=0**. Sau bước khởi tạo, CA UI thể hiện chuỗi Root CA → Intermediate CA.



CA UI hiển thị hai cấp CA

Hình 3: Root CA và Intermediate CA sau khi khởi tạo

3.3 Cấu hình RA

RA quản lý bước đăng ký và cấp chứng chỉ. Luồng EJBCA tạo end entity theo profile đã chọn, nhận CSR rồi phát hành chứng chỉ; việc phê duyệt có thể được chèn giữa các bước theo cấu hình CA [4].

Nhóm tạo certificate profile **TLS-SERVER** và end entity profile **WEB-SERVER**. RA đăng ký **web.local**, kiểm tra tên máy chủ, chọn Intermediate CA và phê duyệt CSR. Mô hình dùng kiểm tra thủ công nên không triển khai ACME challenge.

4 Cấp chứng chỉ HTTPS

Khóa riêng và CSR được tạo trên máy Ubuntu để khóa riêng không rời máy chủ:

```
1 openssl req -new -newkey rsa:2048 -nodes \
2   -keyout web.local.key -out web.local.csr \
3   -subj "/C=VN/O=PKI Lab/CN=web.local" \
4   -addext "subjectAltName=DNS:web.local"
```

RA chọn **Provided by user** cho cấp khóa, nhận CSR và trả chứng chỉ PEM. Tài liệu EJBCA yêu cầu profile máy chủ cho phép **serverAuth** và SAN tương ứng với tên mà client sử dụng [5]. Các tệp đầu vào cho phần tự động hóa gồm:

Tệp	Nội dung
root-ca.crt	Chứng chỉ Root CA
intermediate-ca.crt	Chứng chỉ Intermediate CA
web.local.crt	Chứng chỉ máy chủ do Intermediate CA ký
web.local.key	Khóa riêng của máy chủ

RA UI sau khi phát hành

Hình 4: End entity web.local và chứng chỉ đã cấp

5 Triển khai Nginx

Root CA được chép vào kho tin cậy của Ubuntu. Nginx nhận khóa riêng và tệp chuỗi. Tệp chuỗi hợp lệ đặt chứng chỉ máy chủ trước chứng chỉ Intermediate CA, đúng thứ tự mà Nginx yêu cầu [6].

```
1 cat web.local.crt intermediate-ca.crt > fullchain.pem
2 sudo install -m 0644 root-ca.crt \
3   /usr/local/share/ca-certificates/pki-demo-root.crt
4 sudo update-ca-certificates
```

Ứng dụng chỉ trả một trang HTML ngắn. Nội dung trang không tham gia kiểm thử. Đối tượng kiểm thử là chuỗi chứng chỉ mà Nginx gửi trong TLS handshake.

6 Thiết kế kiểm thử

Mỗi ca kiểm thử dùng hai phép kiểm tra độc lập. OpenSSL dựng đường dẫn từ chứng chỉ đích, danh sách CA trung gian qua **-untrusted** và Root CA tin cậy qua **-CAfile** [7]. curl kết nối đến Nginx và mặc định kiểm tra chữ ký chứng chỉ cùng tên máy chủ theo kho CA của client [8]. Mã thoát của mỗi lệnh là tiêu chí đánh giá chính; thông báo lỗi chỉ hỗ trợ chẩn đoán vì phụ thuộc vào phiên bản thư viện.

Bảng 1: Kết quả mong đợi và tiêu chí đạt

Ca kiểm thử	Điều kiện	OpenSSL	curl
Chuỗi hợp lệ	Leaf và Intermediate nguyên vẹn; Root được tin cậy	0	0
Thiếu Intermediate	Nginx chỉ gửi leaf; OpenSSL không nhận <code>-untrusted</code>	khác 0	khác 0
Sai Issuer	Leaf ghi <code>Intermediate CB</code> ; chữ ký cũ được giữ nguyên	khác 0	khác 0

Ca thiếu Intermediate CA kiểm tra khả năng nổi chứng chỉ máy chủ với Root CA tin cậy. Ở ca sai Issuer, một byte trong `tbsCertificate` được thay đổi. Giá trị trường bị thay đổi từ `Intermediate CA` thành `Intermediate CB`. Sự thay đổi này vừa làm sai liên kết tên vừa làm chữ ký gốc không còn tương ứng với dữ liệu đã ký. Theo RFC 5280, đường dẫn chỉ hợp lệ khi tên issuer khớp và chữ ký được xác minh [1].

Ba ca kiểm thử làm rõ rằng xác thực TLS không chỉ dựa vào chứng chỉ máy chủ. Client phải dựng được đường dẫn liên tục từ Root CA qua Intermediate CA đến `web.local`, đồng thời xác minh dữ liệu trên từng chứng chỉ. Việc thiếu chứng chỉ trung gian hoặc sửa trường `Issuer` đều làm chuỗi không còn đáp ứng các điều kiện này.

7 Kết luận

Phần thực nghiệm đã xác định một quy trình lặp lại được để kiểm tra quan hệ ký trong chuỗi chứng chỉ. Bảng ?? nêu các tiêu chí mong đợi cho từng ca, dựa trên cấu hình đã xác định ở các phần trước.

Ba ca kiểm thử tạo thành một ca đối chứng và hai điều kiện lỗi. Chuỗi đầy đủ cho biết cấu hình tin cậy cơ sở có hoạt động hay không. Việc bỏ Intermediate CA kiểm tra khả năng dựng đường dẫn từ chứng chỉ máy chủ tới Root CA. Ở ca cuối, thay đổi `Issuer` làm sai liên kết tên và thay đổi dữ liệu đã ký trong `tbsCertificate`. Khi OpenSSL và curl trả về mã thoát đúng như bảng ??, chuỗi Root CA → Intermediate CA → `web.local` được xác minh.

Mô hình EJBCA kết hợp với Nginx đã chuyển các yêu cầu của chuỗi chứng chỉ thành các phép kiểm tra cụ thể bằng OpenSSL và curl. Báo cáo khép lại với một kịch bản có thể tái hiện, trong đó mỗi kết quả đều gắn trực tiếp với quan hệ tin cậy giữa ba cấp chứng chỉ.

Tài liệu tham khảo

- [1] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, “Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile”, IETF, RFC 5280, 2008. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5280>
- [2] Keyfactor. “Quick Start Guide – Start EJBCA Container with Client Certificate Authenticated Access”, Accessed: Jul. 23, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/quick-start-guide-start-ejbca-container-with-clien>
- [3] Keyfactor. “Creating a Root CA”, Accessed: Jul. 23, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/creating-a-root-ca>
- [4] Keyfactor. “EJBCA RA Concept Guide”, Accessed: Jul. 23, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/ejbca-ra-concept-guide>
- [5] Keyfactor. “Create Server Certificates”, Accessed: Jul. 23, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/create-server-certificates>

- [6] F5, Inc. “Configuring HTTPS Servers”, Accessed: Jul. 23, 2026. [Online]. Available: https://nginx.org/en/docs/http/configuring_https_servers.html
- [7] OpenSSL Project. “openssl-verify: Certificate Verification Command”, Accessed: Jul. 23, 2026. [Online]. Available: <https://docs.openssl.org/master/man1/openssl-verify/>
- [8] curl project. “TLS Certificate Verification”, Accessed: Jul. 23, 2026. [Online]. Available: <https://curl.se/docs/sslcerts.html>

A Mã kịch bản

Phụ lục dưới đây mô tả chi tiết mã chương trình tự động, dưới dạng Proof-of-Concept (PoC). Mã này mô tả chung kịch bản chạy sau khi RA đã cấp đủ bốn tệp chứng chỉ và khóa.

Listing 1: Kịch bản tự động hóa phần kiểm thử

```

1  #!/usr/bin/env bash
2  set -Eeuo pipefail
3
4  CERT_SOURCE="${1:-$PWD/certs}"
5  DEMO_DIR="${DEMO_DIR:-$PWD/pki-chain-demo}"
6  ROOT_CERT="$CERT_SOURCE/root-ca.crt"
7  INTERMEDIATE_CERT="$CERT_SOURCE/intermediate-ca.crt"
8  LEAF_CERT="$CERT_SOURCE/web.local.crt"
9  LEAF_KEY="$CERT_SOURCE/web.local.key"
10
11 require() {
12     command -v "$1" >/dev/null 2>&1 || {
13         printf 'Missing command: %s\n' "$1" >&2
14         exit 2
15     }
16 }
17
18 for command_name in docker openssl curl perl; do
19     require "$command_name"
20 done
21
22 for input_file in \
23     "$ROOT_CERT" \
24     "$INTERMEDIATE_CERT" \
25     "$LEAF_CERT" \
26     "$LEAF_KEY"; do
27     [[ -f "$input_file" ]] || {
28         printf 'Missing input: %s\n' "$input_file" >&2
29         exit 2
30     }
31 done
32
33 mkdir -p "$DEMO_DIR/certs" "$DEMO_DIR/html"
34 install -m 0600 "$LEAF_KEY" \
35     "$DEMO_DIR/certs/web.local.key"
36 install -m 0644 "$ROOT_CERT" \
37     "$DEMO_DIR/certs/root-ca.crt"
38 install -m 0644 "$INTERMEDIATE_CERT" \
39     "$DEMO_DIR/certs/intermediate-ca.crt"
40 install -m 0644 "$LEAF_CERT" \
41     "$DEMO_DIR/certs/web.local.crt"
42
43 cat >"$DEMO_DIR/html/index.html" <<'HTML'
44 <!doctype html>
45 <html lang="en">
46 <meta charset="utf-8">
47 <title>PKI chain demo</title>
48 <h1>web.local certificate accepted</h1>
49 </html>
50 HTML
51
52 cat >"$DEMO_DIR/nginx.conf" <<'NGINX'
53 server {
54     listen 443 ssl;
55     server_name web.local;
56     ssl_certificate /etc/nginx/certs/fullchain.active.pem;
57     ssl_certificate_key /etc/nginx/certs/web.local.key;

```

```

58     location / {
59         root /usr/share/nginx/html;
60         index index.html;
61     }
62 }
63 NGINX
64
65 cat >"$DEMO_DIR/compose.yaml" <<'YAML'
66 services:
67     nginx:
68         image: nginx:alpine
69         container_name: pki-demo-nginx
70         ports:
71             - "443:443"
72         volumes:
73             - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
74             - ./certs:/etc/nginx/certs:ro
75             - ./html:/usr/share/nginx/html:ro
76 YAML
77
78 if ! grep -Eq \
79     '(~|[:space:~])web\.local([[:space:~]]|$)' \
80     /etc/hosts; then
81     printf '127.0.0.1 web.local\n' |
82         sudo tee -a /etc/hosts >/dev/null
83 fi
84
85 sudo install -m 0644 "$ROOT_CERT" \
86     /usr/local/share/ca-certificates/pki-demo-root.crt
87 sudo update-ca-certificates >/dev/null
88
89 reload_nginx() {
90     docker compose -f "$DEMO_DIR/compose.yaml" \
91         exec nginx nginx -s reload >/dev/null
92 }
93
94 run_case() {
95     local expectation="$1"
96     local label="$2"
97     shift 2
98
99     set +e
100     "$@"
101     local status=$?
102     set -e
103
104     if [[ "$expectation" == success &&
105         "$status" -ne 0 ]]; then
106         printf '[FAIL] %s: expected 0, got %d\n' \
107             "$label" "$status" >&2
108         exit 1
109     fi
110     if [[ "$expectation" == failure &&
111         "$status" -eq 0 ]]; then
112         printf '[FAIL] %s: expected nonzero, got 0\n' \
113             "$label" >&2
114         exit 1
115     fi
116     printf '[PASS] %s: exit %d\n' "$label" "$status"
117 }
118
119 cat "$LEAF_CERT" "$INTERMEDIATE_CERT" \
120 >"$DEMO_DIR/certs/fullchain.valid.pem"
121 cp "$DEMO_DIR/certs/fullchain.valid.pem" \
122     "$DEMO_DIR/certs/fullchain.active.pem"

```



```

123
124 docker compose -f "$DEMO_DIR/compose.yaml" up -d
125
126 run_case success "valid chain / openssl" \
127     openssl verify -CAfile "$ROOT_CERT" \
128         -untrusted "$INTERMEDIATE_CERT" "$LEAF_CERT"
129 run_case success "valid chain / curl" \
130     curl --fail --silent --show-error \
131         --connect-timeout 5 https://web.local/
132
133 cp "$LEAF_CERT" "$DEMO_DIR/certs/fullchain.active.pem"
134 reload_nginx
135
136 run_case failure "missing intermediate / openssl" \
137     openssl verify -CAfile "$ROOT_CERT" "$LEAF_CERT"
138 run_case failure "missing intermediate / curl" \
139     curl --fail --silent --show-error \
140         --connect-timeout 5 https://web.local/
141
142 openssl x509 -in "$LEAF_CERT" -outform DER \
143     -out "$DEMO_DIR/certs/web.local.der"
144 match_count="$(
145     LC_ALL=C grep -ao 'Intermediate CA' \
146         "$DEMO_DIR/certs/web.local.der" |
147     wc -l |
148     tr -d '[:space:]'
149 )"
150 if [[ "$match_count" != 1 ]]; then
151     printf 'Expected one issuer string, found %s\n' \
152         "$match_count" >&2
153     exit 2
154 fi
155
156 LC_ALL=C perl -0pi -e \
157     's/Intermediate CA/Intermediate CB/' \
158     "$DEMO_DIR/certs/web.local.der"
159 openssl x509 -inform DER \
160     -in "$DEMO_DIR/certs/web.local.der" \
161     -out "$DEMO_DIR/certs/web.local.tampered.crt"
162
163 cat "$DEMO_DIR/certs/web.local.tampered.crt" \
164     "$INTERMEDIATE_CERT" \
165     >"$DEMO_DIR/certs/fullchain.tampered.pem"
166 cp "$DEMO_DIR/certs/fullchain.tampered.pem" \
167     "$DEMO_DIR/certs/fullchain.active.pem"
168 reload_nginx
169
170 run_case failure "tampered issuer / openssl" \
171     openssl verify -CAfile "$ROOT_CERT" \
172         -untrusted "$INTERMEDIATE_CERT" \
173         "$DEMO_DIR/certs/web.local.tampered.crt"
174 run_case failure "tampered issuer / curl" \
175     curl --fail --silent --show-error \
176         --connect-timeout 5 https://web.local/
177
178 cp "$DEMO_DIR/certs/fullchain.valid.pem" \
179     "$DEMO_DIR/certs/fullchain.active.pem"
180 reload_nginx
181 printf 'Valid chain restored.\n'

```